Rock, Paper, Scissors, Scalpers:

Trading Card Game + Market Simulator

TEAM 10: Denys Chernenko, Adrian Lozada, Smaran Iyer

Motivation

- Today, many trading card games exist with secondary markets associated with them.
- What if we turned Rock, Paper, Scissors (RPS) into a trading card game?
- RPS is trivially balanced but when **agents** can **buy, sell, and trade cards** on a **market**, interesting things happen.
- ***What happens when you add an economy on top of a simple game?***



Photo Credit: fortune.com





Photo Credit: tcgplayer.com

Project Overview & Game Rules

- **Two agent types:**
  - PLAYER: Compete in RPS matches to win; goal is to **maximize wins.**
  - SCALPER: Do *not* play games; trade cards and **profit from card price fluctuations** in the market
- **Game rules at a glance:**
  - Each player starts with a deck of cards (Rock, Paper, or Scissors)
  - Players are randomly paired; each plays a card; standard RPS resolution
  - After games, players/scalpers can buy/sell cards from a common market
  - Card prices fluctuate based on supply & demand (±5% per tick based on net volume)

Project Overview & Game Rules

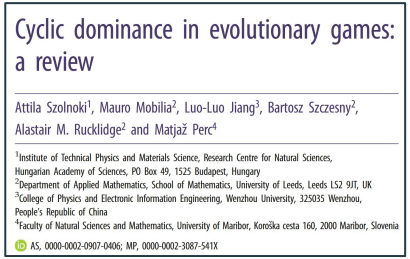
Rule	Detail
Deck	9 cards per player (random R/P/S mix)
Matching	Even number of players paired randomly each tick
Play	Each player shuffles deck → plays top card vs. opponent
Winning	Standard RPS payoff; winner gets a point. First to 3 wins wins the game.
Market	After each round, agents may buy/sell cards; prices adjust ±5% based on net buy/sell pressure

Architecture & Technical Design

In 1 Step:	What it does
matchPlayers()	Shuffle, splits players in half, zips into pairs
buildGames()	Creates a GameOneCard instance per pair
playGames()	Each game resolves according to Game Rules; WIN/LOSS/TIE recorded in player's history
agentUpdate()	Every agent's wake_up_agent(card_prices) generates buy/sell orders; orders batched into the market
marketClears()	Market executes accepted orders, adjusts prices per card based on net demand, settles via callback
recordState()	Snapshot tick state for post-game analysis

Player Strategies

Based on:



Cyclic dominance in evolutionary games: a review

Attila Szolnoki[1], Mauro Mobilia[2], Luo-Luo Jiang[3], Bartosz Szczesny[2], Alastair M. Rucklidge[2] and Matjaž Perc[4]

[1]Institute of Technical Physics and Materials Science, Research Centre for Natural Sciences, Hungarian Academy of Sciences, PO Box 49, 1525 Budapest, Hungary
[2]Department of Applied Mathematics, School of Mathematics, University of Leeds, Leeds LS2 9JT, UK
[3]College of Physics and Electronic Information Engineering, Wenzhou University, 325035 Wenzhou, People's Republic of China
[4]Faculty of Natural Sciences and Mathematics, University of Maribor, Koroška cesta 160, 2000 Maribor, Slovenia

AS, 0000-0002-0907-0406; MP, 0000-0002-3087-541X

Strategy	Behavior	Inspiration	Market Interaction
Mutator	Randomly swaps one card in its deck for a different card type each tick, regardless of game outcomes	Random exploration prevents population collapse by maintaining diversity	Posts one sell order and one buy order each time it fires
Copycat	Compares its own fitness to its opponent's after each game; copies their full deck if the opponent is stronger, with probability proportional to the fitness gap	Social learning successful strategies spread through the population via imitation	Posts one sell order and one buy order each time it fires

Scalper Designs

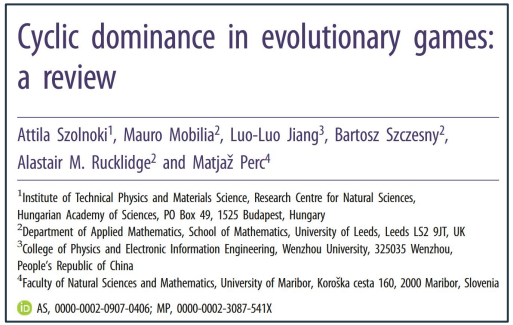
Scalper	Strategy	Entry Logic	Exit Logic
Naive Gambler	Random baseline	Random card, random price [0.5, 2]	Random card, random price
Shadow	EMA trend-filtered mean-reversion	Buy most underpriced card vs EMA, only in uptrend	Trend-reversal / EMA recovery / forced exit (12 ticks)
Quick Grabber	Micro-momentum	Buy card with strongest N-tick price rise above threshold	Profit target (+4%) / stop-loss (−3%) / forced exit (5 ticks)
Market Maker	Fixed-spread mean-reversion	Buy the card with the biggest single-tick decline	Symmetric ±4% spread exit / forced exit (8 ticks)

ODE & Numerical Methods

RPS Population Dynamics ODE System

- **Parameters:**
  - z = copycat rate (default 1.0)
  - μ = mutation rate (default 0.01)
- **Conservation Property**: da/dt + db/dt + dc/dt = 0
- The **ODE** predicts the **expected card population densities** in the absence of a market.
- Comparing ODE trajectories vs. **actual simulation card** distributions reveals how **scalper activity distorts** the natural RPS population dynamic.

Based on:



Cyclic dominance in evolutionary games: a review

Attila Szolnoki[1], Mauro Mobilia[2], Luo-Luo Jiang[3], Bartosz Szczesny[2], Alastair M. Rucklidge[2] and Matjaž Perc[4]

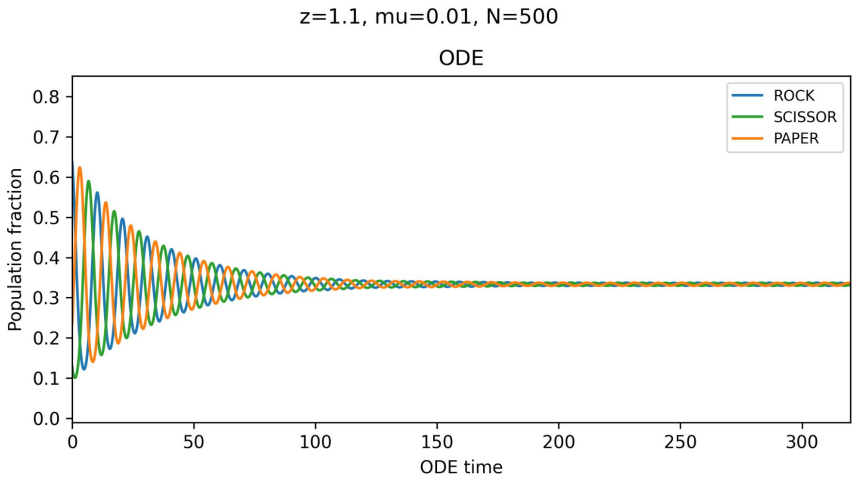
[1]Institute of Technical Physics and Materials Science, Research Centre for Natural Sciences, Hungarian Academy of Sciences, PO Box 49, 1525 Budapest, Hungary
[2]Department of Applied Mathematics, School of Mathematics, University of Leeds, Leeds LS2 9JT, UK
[3]College of Physics and Electronic Information Engineering, Wenzhou University, 325035 Wenzhou, People's Republic of China
[4]Faculty of Natural Sciences and Mathematics, University of Maribor, Koroška cesta 160, 2000 Maribor, Slovenia

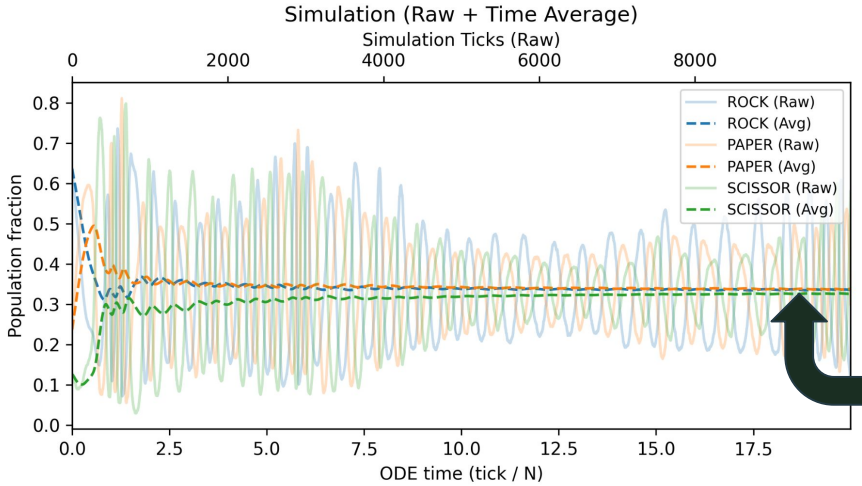
AS, 0000-0002-0907-0406; MP, 0000-0002-3087-541X

$$\frac{da}{dt} = a \cdot z(b - c) + \mu(b + c - 2a)$$

$$\frac{db}{dt} = b \cdot z(c - a) + \mu(a + c - 2b)$$

$$\frac{dc}{dt} = c \cdot z(a - b) + \mu(a + b - 2c)$$





Baseline ODE Trajectory

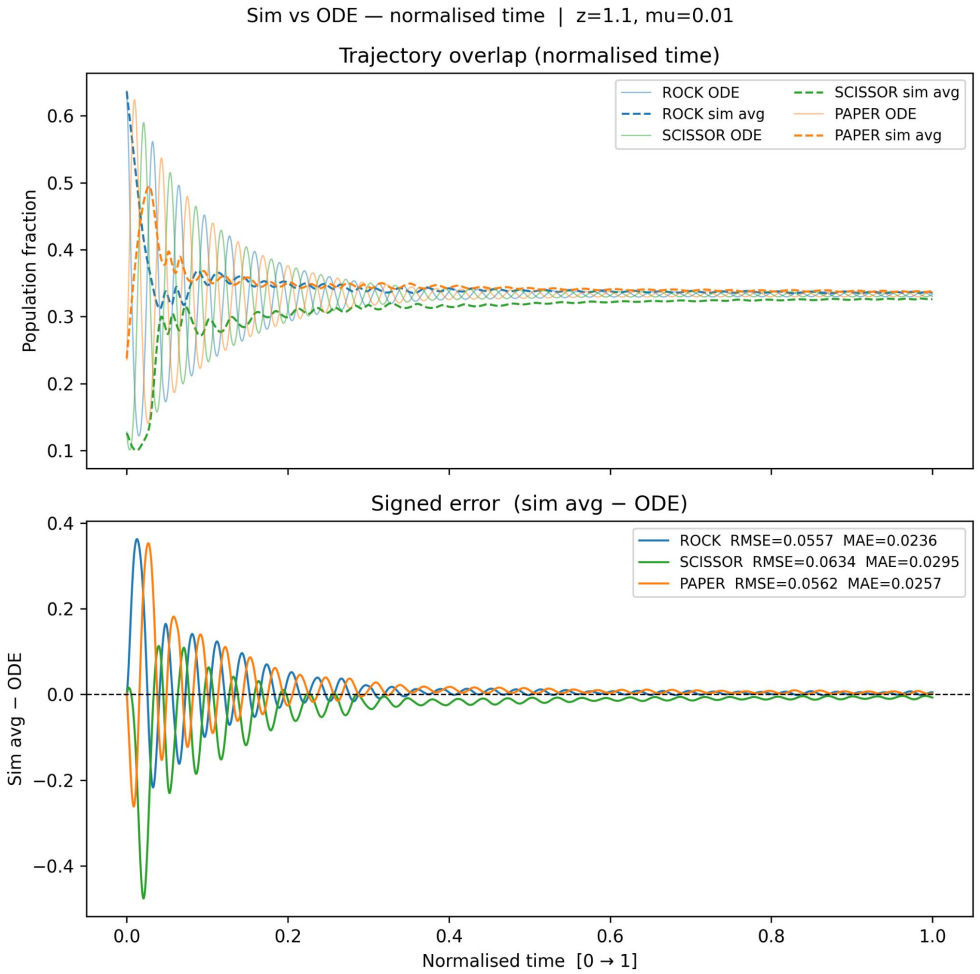
Baseline = No Market, No Scalpers, Less Noise

This baseline converges to R = P = S ≈ 0.333…

Agent Simulation Trajectory

With the same starting parameters, our simulation converges similarly to the baseline ODE

The raw simulation is noisy, but the cumulative mean populations are stable, and tracks with the Baseline Trajectory



Baseline ODE Trajectory

Numerical Validation:

- Root Mean Squared Error
- Mean Absolute Error

$$\mathrm{RMSE} = \sqrt{\frac{1}{K}\sum_{k=1}^{K}\left(\bar{x}_k - x_k^{\mathrm{ODE}}\right)^2},$$

$$\mathrm{MAE} = \frac{1}{K}\sum_{k=1}^{K}\left|\bar{x}_k - x_k^{\mathrm{ODE}}\right|.$$

Card	RMSE	MAE	Max |err|
Rock	0.0557	0.0236	0.3626
Scissor	0.0634	0.0295	0.4762
Paper	0.0562	0.0257	0.3531

Very close to baseline = Good sign!

Conclusions

What we built:

- A full **agent-based simulation** coupling RPS competition with a **card market.**
- 2 Player Strategies
- **4 distinct scalper strategies** of varying sophistication
- Numerically validated **Analytical ODE baseline** grounded in published research

Key Results / Takeaways:

- Our **numerical validation** indicates our simulation appears to be **unbiased, implemented properly**
- ODE baseline confirms that **without market intervention**, card populations should **converge to expected proportions.**
- Trend-aware scalpers (non-Naive Gambler) **outperform baseline** (Naive Gambler) significantly

Future Work

- **Further Experimentation**: We built a simulation, but what can it tell us about **secondary card markets?** More testing is required.
- **Speed**: The current implementation is naive and could be much more performant.
- **Reporting**: Build **post-simulation analytics** and CSV export for **data analysis**
- **UI Integration**: Currently we are using the terminal. We can build an **interactive dashboard** for better **visualization.**